

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	DSA0307	Course Title:	Natural Language Processing
CO Assessed:	CO4 – Precision	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	20% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	Charandeep.N.C	Reg. No.:	192472196
Section / Batch:	CSE-AI	Date:	

Code of Conduct

I Charandeep.N.C (192472196) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: *Charandeep*

Instructions

- Read each scenario carefully before answering.
- Identify the NLP techniques being compared in the question.
- Analyze each approach based on its working principles, advantages, limitations, and applicability.
- Compare the alternatives using technical reasoning rather than listing definitions.
- Support your analysis with examples from the given scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze sentence representations, compare structural representations of language, and evaluate how different parsing approaches capture meaning and relationships among words.	Q1
Syntax-Driven Semantic Analysis	Analyze parsing strategies for incomplete and ambiguous inputs, evaluate parsing efficiency, and determine suitable methods for real-time language understanding.	Q2
Lexemes and Their Senses & Word Sense Disambiguation	Analyze ambiguity in natural language sentences, evaluate ambiguity resolution techniques, and determine the most effective approach for selecting intended meanings.	Q3
Representing Linguistically Relevant Concepts	Analyze grammatical constraints, agreement features, argument structures, and linguistic representations required for accurate language processing.	Q4
Information Retrieval & Syntax-Driven Semantic Analysis	Analyze large-scale parsing strategies, evaluate performance trade-offs, and justify efficient approaches for processing massive linguistic datasets.	Q5

Annexure A – Comparative Analysis

1. A language processing system must represent sentence structure for grammar checking and syntactic analysis. The developers are deciding between Context-Free Grammar (CFG) trees and dependency parsing structures. Analyze how these two approaches differ in representing sentence structure and justify which is more effective for capturing relationships between words.

Introduction

Representing the structure of a sentence is the foundation on which every downstream language-processing task — grammar checking, syntactic analysis, machine translation, information extraction — is built. When a development team has to choose a representation scheme, the decision is not merely academic: it determines how easily the system can detect grammatical errors, how directly it can expose relationships between words, and how well it scales to the free word order and long-distance dependencies found in real text. The two candidate representations under consideration here — Context-Free Grammar (CFG) parse trees (constituency structures) and dependency parsing structures — encode the same underlying sentence in fundamentally different ways. CFG trees group words into nested phrasal constituents (Noun Phrases, Verb Phrases, Prepositional Phrases, and so on), while dependency structures connect words directly to one another through labelled, directed grammatical relations such as subject, object, and modifier. This answer analyses both approaches on their working principles, representational power, strengths, limitations, and suitability for grammar checking, and then justifies which of the two is more effective at capturing relationships between words.

Approach 1: Context-Free Grammar (Constituency) Trees — Working Principle

A CFG defines a sentence recursively through a set of production rules of the form $A \rightarrow \beta$, where A is a non-terminal symbol (a phrase category such as S , NP , VP , PP) and β is a sequence of terminals and/or non-terminals. Parsing a sentence with a CFG means finding a derivation tree that starts from the start symbol S and, through repeated rule application, rewrites down to the actual words (terminals) of the sentence. For the sentence "The cat chased the mouse," a small CFG might contain the rules $S \rightarrow NP VP$, $NP \rightarrow Det N$, $VP \rightarrow V NP$, $Det \rightarrow \text{"the"}$, $N \rightarrow \text{"cat"} \mid \text{"mouse"}$, $V \rightarrow \text{"chased"}$. Applying these rules produces a tree in which S branches into an NP ("The cat") and a VP ("chased the mouse"), and the VP itself branches further into a V ("chased") and an NP ("the mouse").

The resulting structure is a constituency tree: every internal node represents a phrase, and every phrase is a constituent that can, in principle, be replaced as a unit (e.g., "the mouse" can be replaced by "it" without breaking the grammaticality of the sentence, which is exactly the substitution test linguists use to justify constituency). Because CFG trees are built from rewrite rules, they naturally encode phrase-level grammaticality: a sentence is syntactically well-formed with respect to the grammar if and only if a complete derivation exists. This makes CFGs a natural fit for tasks that need to validate whether a sentence conforms to the phrase-structure rules of a language, such as checking that a Noun Phrase is correctly formed or that a Verb Phrase has the expected number of arguments in the correct linear positions.

Approach 2: Dependency Parsing Structures — Working Principle

Dependency parsing abandons the notion of intermediate phrasal nodes altogether. Instead, every word in the sentence becomes a node in a tree, and a directed, labelled arc is drawn from a "head" word to each of its "dependents." For "The cat chased the mouse," a dependency parser would produce: $\text{chased} \rightarrow \text{cat}$ (nsubj, the nominal subject of chased), $\text{chased} \rightarrow \text{mouse}$ (dobj, the direct object of chased), $\text{cat} \rightarrow \text{the}$ (det), $\text{mouse} \rightarrow \text{the}$ (det). The verb "chased" becomes the root of the tree because every other word in the

sentence is, directly or transitively, dependent on it. There is no NP or VP node; the grammatical relation is attached directly to the word pair via the arc label.

Dependency grammar is built on the linguistic assumption that syntax is fundamentally about relations between individual words (head-dependent relations) rather than about groupings of words into constituents. This design has an important practical consequence: because the arc directly connects a governor to its dependent and is labelled with the grammatical function (subject, object, modifier, determiner, and so on), the relationship between any two words in the sentence can be read off the tree in a single step, without having to walk up and down a constituency hierarchy to determine, for instance, which NP is the subject of which VP.

Comparative Analysis

Representational focus. CFG trees describe how a sentence is built up out of phrases; dependency trees describe how words govern other words. The former answers "what phrase is this?" while the latter answers "what is this word's grammatical role, and with respect to which head?"

Directness of word relationships. In a CFG tree, to find out whether "mouse" is the object of "chased," a grammar checker has to traverse from the VP node down to its child NP and compare it against the position and category of "chased." In a dependency tree the same fact is a single labelled edge: chased — dobj → mouse. This makes dependency structures considerably more direct for exactly the kind of relational queries a grammar checker or a semantic-role labeller needs to answer.

Handling of free word order. Languages with relatively fixed word order (English, to a good approximation) are described comfortably by CFGs because phrase order is largely fixed by the rules. Languages with freer word order (Czech, Turkish, Sanskrit) are far more naturally handled by dependency grammar, because the head-dependent relation between "subject" and "verb" does not change even if their linear position in the sentence changes. Grammar-checking systems that need to be robust to reordering, ellipsis, and scrambled or spoken-style input therefore benefit from a dependency representation.

Ambiguity and grammar size. CFGs require an explicit, often large, rule set to cover the phrase types of a language, and structural ambiguity (multiple legal derivations for one sentence) grows quickly as the grammar is enlarged to cover more constructions. Dependency grammars, especially data-driven ones trained on treebanks (e.g., Universal Dependencies), sidestep much of this because the parser is learning head-selection decisions rather than enumerating phrase-structure rules, which tends to generalize better to constructions not explicitly anticipated by a rule-writer.

Detecting specific grammar errors. Many common grammar errors are essentially local agreement or argument-structure errors: "The dogs barks," "She go to school," "He gave the book to." These are precisely subject-verb agreement, subject-verb, and argument-omission relations — exactly the kind of head-dependent pair that dependency parsing exposes directly. A CFG-based checker would have to descend through the NP and VP nodes and separately verify number agreement between the head noun of the subject NP and the head verb of the VP, which is a more indirect, multi-step check.

Interpretability and downstream use. Constituency trees are more convenient when the downstream task genuinely needs phrase boundaries — for example, chunking text into Noun Phrases for a search index, or feeding a syntax-based machine-translation system that reorders whole constituents. Dependency trees are more convenient when the downstream task needs semantic-role-like information — for example, information extraction, relation extraction, and question answering, where "who did what to whom" is the target information, which maps almost one-to-one onto dependency arc labels (nsubj, dobj, iobj).

Example Illustration

Consider the grammar-checking scenario directly. Given the sentence "The children plays in the park," a CFG-based system would parse the sentence into $S \rightarrow NP VP$, with $NP \rightarrow$ "The children" and $VP \rightarrow$ "plays in the park." To detect the number-agreement error, the checker must locate the head noun inside the NP ("children," plural) and the head verb inside the VP ("plays," singular) and compare their morphological features — a check that spans two different subtrees and requires the system to know which node inside each phrase is the "head" in the first place, information that is not explicit in a plain CFG tree unless head-marking is added on top of it (as in head-driven or lexicalized CFGs).

A dependency parser, in contrast, directly produces the arc $\text{plays} \xrightarrow{\text{nsubj}} \text{children}$. Because the arc already identifies "children" as the subject of "plays," an agreement checker only needs to compare the grammatical-number feature of these two words, without any additional traversal or head-finding logic. The relationship the checker cares about — subject-verb agreement — is exactly the relationship the dependency structure encodes as a first-class edge.

Justification and Conclusion

Both representations are formally capable of describing the syntax of a sentence, and in fact a dependency tree can, for projective structures, be derived from a lexicalized CFG tree and vice versa — the two are not mutually exclusive but are different views optimized for different questions. However, when the specific requirement is to capture relationships between words — which is precisely what grammar checking and syntactic error detection need — dependency parsing structures are more effective. They make the head-dependent relation, together with its grammatical label, an explicit, directly queryable edge in the graph, rather than an implicit fact that has to be reconstructed by traversing and interpreting a phrase-structure hierarchy. Dependency structures are also more robust to word-order variation, easier to train from annotated treebanks without hand-writing a large rule set, and align more naturally with the "who depends on whom, and how" questions that agreement checking, argument-structure checking, and relation extraction all ultimately ask. CFG/constituency trees remain valuable when the task genuinely needs phrase-level groupings (for chunking, constituent-based reordering in translation, or syntax-directed compilation of natural-language commands), so a mature grammar-checking system may still use shallow phrase chunking as a preprocessing step. But as the core representation for capturing the grammatical relationships between individual words, the dependency parsing structure is the more effective and more directly usable choice.

2. A real-time application needs to parse user input efficiently, even when sentences are incomplete or ambiguous. The system currently uses top-down parsing, but an alternative is Earley parsing. Analyze the differences between these parsing techniques and reason which is more suitable for such dynamic input conditions.

Introduction

Real-time language-understanding systems — voice assistants, live chat interfaces, incremental speech recognizers, and autocomplete engines — face a parsing problem that batch NLP systems do not: the input may be incomplete at the moment parsing begins, and it is very often ambiguous even once complete. The parser must therefore be efficient, must be able to work with partial input, and must handle ambiguity gracefully rather than failing outright. The system under discussion currently uses top-down parsing and is considering a switch to Earley parsing. This answer analyses the working principles of both techniques, compares them along the dimensions that matter for dynamic, real-time input — efficiency, handling of

ambiguity, and handling of incomplete or partial sentences — and justifies which is the more suitable choice.

Approach 1: Top-Down Parsing — Working Principle

Top-down parsing begins at the start symbol of the grammar (S) and attempts to derive the input sentence by expanding non-terminals according to the grammar's production rules, working downward until the leaves of the derivation match the actual words of the sentence. For example, given $S \rightarrow NP VP$, the parser first predicts an NP, tries to expand it using the available NP rules ($NP \rightarrow Det N$, $NP \rightarrow Pronoun$, etc.), and checks whether the next input token is consistent with the chosen expansion. If it is not, the parser must backtrack and try an alternative rule.

This "predict, then match, then backtrack on failure" strategy has two well-known weaknesses. First, in its naive form top-down parsing cannot handle left-recursive grammar rules (a rule such as $NP \rightarrow NP PP$ would cause the parser to expand $NP \rightarrow NP PP \rightarrow NP PP PP \dots$ forever without ever consuming an input token), so grammars have to be rewritten to avoid left recursion before top-down parsing can be applied. Second, and more importantly for a real-time system, the algorithm explores each candidate expansion largely independently: when several rules could apply, the parser must try one, and if that path eventually fails deep into the sentence, it backtracks all the way and retries a different top-level choice. In the worst case, this exploration is exponential in sentence length, because the same substring of the input may be re-parsed multiple times under different guesses about the phrases above it — there is no general mechanism for sharing partial-parse work across different backtracking branches.

Approach 2: Earley Parsing — Working Principle

The Earley algorithm is a chart-parsing technique that processes the input left to right and, for every position i in the sentence, maintains a "state set" (or "chart entry") $S(i)$ containing all partial parses ("dotted rules," e.g., $NP \rightarrow Det \bullet N$) that are valid given the input consumed so far. At each step, the algorithm performs three operations: (1) prediction — for every incomplete rule in the current state set whose next expected symbol is a non-terminal, add all the grammar rules that could expand that non-terminal; (2) scanning — for every incomplete rule whose next expected symbol matches the next input word, advance the dot past that word and add the resulting state to the next state set; (3) completion — whenever a rule is fully matched (dot at the end), propagate that fact back to any state in an earlier state set that was waiting for exactly this constituent, advancing its dot.

Crucially, Earley parsing never throws away and re-derives a sub-parse it has already computed: once a constituent spanning positions i to j has been recognized, that fact is recorded in the chart and reused by every other partial parse that needs a constituent of that type over that span. This dynamic-programming property gives Earley parsing a worst-case time complexity of $O(n^3)$ for arbitrary context-free grammars (and much better, close to linear, for grammars that are not heavily ambiguous), compared to the potentially exponential blow-up of a naive backtracking top-down parser. Earley parsing also handles left-recursive rules directly, with no grammar transformation required, because prediction and completion do not depend on ever finishing the derivation before making the next move.

Comparative Analysis

Handling ambiguity. Top-down parsing with backtracking explores ambiguous analyses one at a time, discarding all state from a path the moment it backtracks; if there are multiple valid parses, they must be found in separate, largely repeated, passes. Earley parsing represents all valid partial analyses compactly in the chart simultaneously — ambiguity is represented as multiple dotted-rule entries in the same state

set, not as separate search branches — so ambiguous input does not force repeated re-parsing of shared substructure. For sentences that are genuinely ambiguous (which spoken or casually typed real-time input very often is), this is a decisive advantage.

Handling incomplete input. This is the most important distinction for the stated use case. Because Earley parsing builds the chart incrementally, one word at a time, the state set $S(i)$ after processing the first i words is always a well-defined, valid parsing state — it directly represents "everything the grammar allows to be true given exactly these i words." This means an Earley parser can be paused after any prefix of the sentence and can report, from the current chart, which continuations are grammatically possible, which is exactly what a real-time system needs when the user is still typing or speaking. Top-down backtracking parsing, by contrast, is oriented around trying to reach a complete derivation for the whole sentence; if the input is cut off mid-sentence, the parser is typically left in the middle of a partially explored, partially backtracked search that does not correspond to a clean, well-defined "state as of word i " — extracting a useful partial result requires extra engineering that is not native to the algorithm.

Time complexity and predictability. Naive top-down backtracking parsing has no polynomial worst-case guarantee for ambiguous or complex grammars — pathological sentences can cause combinatorial blow-up, which is unacceptable in a system that must respond within a fixed real-time budget (e.g., under 100ms for a voice interface). Earley parsing guarantees $O(n^3)$ in the worst case and performs close to $O(n)$ on grammars typical of restricted-domain command grammars, giving the system a predictable, boundable response time as sentence length grows — an important property for real-time systems that must meet latency targets rather than just average-case speed.

Memory/state sharing. Earley's chart explicitly shares sub-derivations across ambiguous analyses (a form of dynamic programming similar in spirit to CYK parsing), whereas top-down backtracking recomputes shared substructure repeatedly across different backtracking branches, wasting work precisely on the over-lapping, common parts of alternative analyses — which are often the majority of the sentence when only one attachment decision differs between two readings.

Grammar restrictions. Top-down parsing (without transformation) cannot use left-recursive rules at all, which restricts how naturally certain linguistic constructions (e.g., possessive/recursive NPs, "the dog's owner's house") can be written in the grammar. Earley parsing places no such restriction on the grammar, giving grammar designers more freedom and reducing the risk of subtle bugs introduced by grammar rewriting.

Example Illustration

Consider a voice-assistant command such as "Set a reminder for tomorrow at" spoken with the user pausing mid-utterance (perhaps due to hesitation, background noise cutting the audio, or the system needing to respond before the utterance is complete). With Earley parsing, the chart after the word "at" already encodes that the grammar expects a time expression next (because the PP/time-argument rule for "reminder for <date> at <time>" has been predicted and partially scanned); the system can use this directly to prompt "At what time?" or to trigger a time-entry UI. With top-down backtracking parsing, the parser was in the middle of trying to derive a full sentence, and if it had already backtracked out of one candidate rule for "reminder" while trying another when the input stopped, there may be no single coherent partial state to read a suggestion from — the system would need bespoke, non-standard bookkeeping bolted onto the top-down algorithm to recover this information, whereas it is a natural byproduct of how Earley parsing already works.

Now consider a genuinely ambiguous typed query such as "Book a flight to Boston from New York on Friday," where "on Friday" could in principle attach to "Boston," to "New York," or to the whole booking event. A top-down parser exploring PP-attachment choices would try one attachment, and if a later grammar or semantic check rejected it, would backtrack and retry the others sequentially, redoing the shared work of parsing "Book a flight to Boston from New York" each time. An Earley parser keeps all attachment possibilities live in the same chart, sharing the already-parsed common prefix, and lets a later disambiguation stage (statistical scoring, semantic filtering) choose among the alternatives without re-parsing.

Justification and Conclusion

For a real-time application that must parse potentially incomplete and ambiguous user input, Earley parsing is clearly the more suitable technique. Its incremental, left-to-right chart construction produces a well-defined parsing state after every input token, which is exactly what is needed to support partial input, live suggestions, and graceful handling of interruption. Its polynomial worst-case time bound and its dynamic-programming sharing of common sub-derivations make its performance predictable and efficient even on ambiguous grammars, avoiding the exponential blow-up and repeated recomputation that naive top-down backtracking parsing is prone to. Top-down parsing remains attractive for its conceptual simplicity and its direct correspondence to hand-written recursive-descent implementations when the grammar is small, unambiguous, and known to be free of left recursion (many simple command grammars for constrained-domain assistants are written this way for exactly that reason). But once the system must be robust to the messiness of real, dynamic input — hesitations, partial utterances, and structurally ambiguous phrasing — Earley parsing's incremental chart and guaranteed polynomial complexity make it the technically superior and more suitable choice.

3. An NLP model must handle ambiguity in sentences such as “She saw the man with a telescope.” The designers are considering CFG, PCFG, and neural parsing techniques. Analyze how these approaches differ in handling ambiguity and reason which method is most effective for real-world applications.

Introduction

Ambiguity is arguably the central difficulty in natural language parsing. The classic example sentence "She saw the man with a telescope" is structurally ambiguous: "with a telescope" can attach to the verb "saw" (she used a telescope to see him) or to the noun phrase "the man" (the man she saw happened to have a telescope). A parser has to decide, implicitly or explicitly, which of these readings is intended, and different parsing paradigms handle this decision very differently. This answer analyses three approaches — plain Context-Free Grammar (CFG) parsing, Probabilistic Context-Free Grammar (PCFG) parsing, and neural parsing — in terms of how each one handles ambiguity such as prepositional-phrase (PP) attachment, and reasons about which is most effective for real-world applications.

Approach 1: Plain CFG Parsing — Working Principle and Handling of Ambiguity

A plain CFG parser (using, for example, CYK or Earley parsing) finds every derivation of the sentence that is licensed by the grammar rules. For "She saw the man with a telescope," a grammar that allows both $VP \rightarrow V NP PP$ and $NP \rightarrow NP PP$ will produce two distinct, equally "grammatical" parse trees — one where the PP attaches under the VP (verb attachment) and one where it attaches under the object NP (noun attachment). Crucially, a plain CFG has no built-in mechanism to prefer one of these parses over the other: both are syntactically legal with respect to the grammar, so the parser returns both (or, if implemented to

return a single answer, an arbitrary one determined by rule ordering or search strategy) without any notion of which is more likely to be the intended meaning. Disambiguating between them requires bolting on separate, hand-crafted heuristics or rules external to the grammar itself (e.g., "always prefer noun attachment," or manually written selectional-restriction rules that check whether "telescope" is a plausible instrument for "seeing"). This makes plain CFG parsing structurally correct — it correctly identifies that the sentence is ambiguous and enumerates the legal readings — but practically incomplete, because it provides no principled, general way to resolve the ambiguity.

Approach 2: PCFG Parsing — Working Principle and Handling of Ambiguity

A Probabilistic CFG augments every production rule with a probability estimated from a treebank corpus, such that the probabilities of all rules expanding a given non-terminal sum to 1. Given a sentence, a PCFG parser (typically using a probabilistic version of CYK, sometimes called the Viterbi/CKY algorithm for PCFGs) computes the probability of every possible derivation as the product of the probabilities of the rules used, and returns the single derivation with the highest overall probability. For the telescope sentence, if the training corpus contains many more examples of PPs attaching to verbs of perception with instrument-like PPs than to their object nouns (or vice versa), the rule probabilities will reflect that statistical tendency, and the PCFG parser will automatically prefer the higher-probability attachment without needing a hand-written disambiguation rule.

This is a genuine improvement over plain CFG parsing: ambiguity is resolved automatically and the resolution is grounded in corpus statistics rather than manual heuristics. However, PCFGs base their probability purely on the syntactic rule used (e.g., how often $VP \rightarrow V\ NP\ PP$ occurs versus $NP \rightarrow NP\ PP$ occurs), independent of the actual words filling those slots. This means a PCFG cannot easily distinguish "she saw the man with a telescope" (instrument reading more plausible) from "she saw the man with a scar" (possessive/descriptive reading far more plausible, since "scar" is not an instrument for seeing), because both instantiate the same rule-level ambiguity ($VP \rightarrow V\ NP\ PP$ vs. $NP \rightarrow NP\ PP$) and would receive the same rule probabilities regardless of the specific lexical items "telescope" or "scar." Some improvement is possible through lexicalized PCFGs, which condition rule probabilities on head words, but this comes at a significant cost in the number of parameters that must be estimated and the amount of training data required.

Approach 3: Neural Parsing — Working Principle and Handling of Ambiguity

Neural parsers (e.g., neural transition-based parsers, neural constituency parsers using recurrent or transformer encoders, and neural dependency parsers built on top of contextual embeddings such as BERT) replace hand-designed grammar rules and rule-level probabilities with learned, continuous vector representations of words and their context. Every word is embedded not just according to its identity but according to the full sentence context around it, so the word "telescope" in "she saw the man with a telescope" is represented differently depending on the surrounding words, and the model has, in principle, access to the full lexical-semantic content of the sentence (that "saw" is a perception verb, that "telescope" is an optical instrument commonly associated with "seeing," and so on) when it scores possible attachments.

Neural parsers resolve the PP-attachment decision (and ambiguity more generally) as part of a single, jointly trained scoring function over the whole sentence rather than as a product of independently estimated context-free rule probabilities. Because the model is trained end-to-end on large treebanks (and, in modern systems, benefits from pretraining on enormous unlabelled text corpora that encode broad world and lexical knowledge), it can implicitly learn the kind of semantic/selectional preferences that a plain

PCFG cannot represent — for instance, that "telescope" is plausible as an instrument of "see" while "scar" is not, purely from having seen many contextual usages of both words during pretraining. This gives neural parsers meaningfully higher disambiguation accuracy on genuinely ambiguous constructions such as PP-attachment, coordination scope, and relative-clause attachment, which is exactly the class of ambiguity illustrated by the telescope sentence.

Comparative Analysis

Basis of disambiguation. Plain CFG: none (returns all legal parses or an arbitrary one). PCFG: syntactic rule frequency only. Neural parsing: learned representation that can capture lexical-semantic and broad contextual cues.

Accuracy on real ambiguous sentences. Plain CFG parsers, when forced to output a single parse, are essentially guessing; PCFGs improve on this by exploiting statistical regularities but plateau because they largely ignore lexical semantics unless heavily lexicalized (which is expensive); neural parsers achieve the highest reported parsing accuracies on standard benchmarks (e.g., the Penn Treebank) precisely because they can use lexical and contextual information that plain and moderately lexicalized PCFGs cannot easily represent.

Data and computational requirements. Plain CFGs need only a grammar (no training data for probabilities, though a grammar still has to be written or extracted). PCFGs need a treebank to estimate rule probabilities, which is comparatively lightweight to train and fast to parse with. Neural parsers need substantial labelled treebank data (and, in practice, benefit enormously from large-scale pretraining) and require considerably more computational resources both to train and, to a lesser extent, to run inference, especially if built on large transformer encoders.

Interpretability and error analysis. Plain CFG and PCFG derivations are transparent: every decision corresponds to an explicit, human-readable grammar rule and an associated probability, which makes it easy for a linguist or engineer to see exactly why a particular parse was chosen and to correct the grammar or the corpus. Neural parsers are comparatively opaque; the specific reason a model preferred one attachment over another is distributed across learned weights and is not directly inspectable, which complicates debugging and error correction in production systems.

Robustness to novel or out-of-domain language. Plain CFGs are brittle outside the exact coverage of their hand-written rules. PCFGs generalize somewhat better within the syntactic patterns seen in their training treebank but still struggle with unfamiliar rule combinations. Neural parsers, especially those built on pretrained language models, generalize considerably better to informal, out-of-domain, or previously unseen phrasing, because their representations are learned from very broad data rather than a fixed rule inventory.

Example Illustration

Applying all three approaches to "She saw the man with a telescope": a plain CFG parser using a grammar that permits both VP-attachment and NP-attachment of the PP will simply output two parse trees with no ranking between them, leaving the ambiguity fully unresolved for any downstream component. A PCFG trained on a general treebank will likely favor whichever attachment pattern (verb- or noun-attached PP) is more frequent across all PPs in the training data, a decision that is not sensitive to the fact that "telescope" specifically signals an instrument-of-seeing reading — so the PCFG might get this particular sentence right "by accident" (because verb attachment for instrument PPs happens to be the more frequent pattern generally) but would be equally likely to mis-attach a similarly structured sentence where the

correct reading runs against the general frequency trend. A neural parser conditioned on contextual embeddings is much more likely to correctly favor the verb-attachment ("saw...with a telescope" = instrument) reading specifically because "telescope" and "saw" co-occur with strong instrument-of-perception associations in the very large corpora typically used for pretraining, giving the model access to fine-grained lexical evidence that neither the plain CFG nor the standard PCFG can use.

Justification and Conclusion

All three techniques agree on what the possible parses of the ambiguous sentence are — this is determined by the grammar's coverage, not by the disambiguation method. What differs sharply is each technique's ability to select the correct reading. Plain CFG parsing provides no disambiguation mechanism at all and is therefore the weakest of the three for real-world use, since production systems cannot present users with an unranked list of alternative readings for every ambiguous sentence. PCFG parsing is a meaningful improvement, providing automatic, corpus-grounded disambiguation, but is fundamentally limited by its reliance on rule-level (largely non-lexical) probabilities, which prevents it from reliably capturing the lexical-semantic cues that actually determine the correct reading in cases like PP-attachment. Neural parsing is the most effective approach for real-world applications because it learns disambiguation jointly with a rich, contextual representation of meaning, achieving higher accuracy on exactly the kind of structurally ambiguous, lexically sensitive sentences that plain CFG and PCFG approaches struggle with, and it also generalizes better to the varied, noisy, and previously unseen phrasing typical of real-world text and speech. The trade-off is reduced interpretability and higher data/compute cost, which is why PCFG and rule-based approaches are still used in constrained, resource-limited, or highly interpretable domains (e.g., some grammar-checking pipelines and controlled-language systems); but for open-domain, real-world natural language understanding, neural parsing is the most effective method for handling ambiguity.

4. A grammar system needs to correctly handle subject–verb agreement and verb argument structures. The developers are comparing feature structures and subcategorization frames. Analyze how these approaches differ and justify which provides better support for enforcing grammatical correctness.

Introduction

Enforcing grammatical correctness requires a representation that can express two distinct, but related, kinds of linguistic constraints. The first is agreement — the requirement that certain grammatical features (number, person, gender, tense) must match between related words, such as a subject and its verb ("the dog barks" versus "the dogs bark"). The second is argument structure — the requirement that a verb (or other predicate) combine with the correct number and type of complements, such as the fact that "put" obligatorily requires both a direct object and a locative phrase ("put the book on the table," not "put the book"). The grammar system under discussion is comparing feature structures and subcategorization frames as the mechanism for representing these constraints. This answer analyses how the two approaches differ, and justifies which provides better support for enforcing grammatical correctness overall.

Approach 1: Feature Structures — Working Principle

A feature structure is a set of attribute–value pairs (often represented as an attribute-value matrix, or AVM) attached to a linguistic constituent, where the values may themselves be atomic (e.g., NUMBER = singular) or nested feature structures, and where two feature structures can be combined via a unification operation. For example, the word "dog" might carry the feature structure [CAT = N, NUMBER = singular,

PERSON = 3rd], while "dogs" carries [CAT = N, NUMBER = plural, PERSON = 3rd]. A grammar rule for subject–verb agreement can then require that the NUMBER feature of the subject NP unify with the NUMBER feature of the verb: a rule such as $S \rightarrow NP VP$, with the constraint $NP.NUMBER = VP.NUMBER$, will succeed only if the two feature structures are unifiable (i.e., they do not assign conflicting values to a shared attribute). If the subject is [NUMBER = plural] and the verb is [NUMBER = singular] (as in "the dogs barks"), unification fails, and the rule application is rejected — the grammar formally detects the agreement violation as a unification failure.

This mechanism generalizes to any set of morphosyntactic features, not just number: person, gender, case, tense, and even more abstract features can all be represented as attributes with values, and multiple agreement constraints (e.g., number and person simultaneously) can be checked in the same unification step. Feature structures are the representational backbone of unification-based grammar formalisms such as Generalized Phrase Structure Grammar (GPSG), Head-Driven Phrase Structure Grammar (HPSG), and Lexical-Functional Grammar (LFG, via f-structures), precisely because they provide a uniform, compositional way to propagate and check grammatical features across a derivation.

Approach 2: Subcategorization Frames — Working Principle

A subcategorization frame is a lexical specification, attached to a predicate (typically a verb), that lists the syntactic arguments (complements) that predicate obligatorily or optionally takes. For example, "devour" might carry the subcategorization frame [$_ NP$] (transitive: must take a direct object NP — "devour the cake," not "*devour"), "give" might carry [$_ NP NP$] or [$_ NP PP$] (ditransitive: "give her the book" / "give the book to her"), and "sleep" might carry [$_$] (intransitive: no complement required, "*sleep the bed" is ill-formed). Subcategorization frames are typically stored in the lexicon, one (or more, for verbs with multiple valid frames) per predicate, and the parser checks, at the point where a VP is built from a verb and its candidate complements, that the actual complements present match one of the frames licensed for that verb. If a verb like "devour" is used without any object, the subcategorization check fails, and the parser correctly rejects the sentence as ill-formed, or flags a missing-argument grammatical error.

Subcategorization is fundamentally about argument structure and valency — how many arguments a predicate needs and what syntactic category/case each argument must have — rather than about morphosyntactic agreement between independently inflected words.

Comparative Analysis

What each mechanism actually constrains. Feature structures constrain agreement between words that must share compatible grammatical features (subject–verb number/person agreement, determiner–noun agreement, adjective–noun gender agreement in languages that have it). Subcategorization frames constrain the presence, count, and category of a predicate's complements (transitive vs. intransitive vs. ditransitive usage, the choice between an NP object and a PP object, and so on). These are genuinely different classes of grammatical correctness: a sentence can violate agreement while having entirely correct argument structure ("the dog bark" — correct valency for "bark," wrong number agreement), and a sentence can violate argument structure while having internally consistent agreement ("she sleeps the bed" — "sleeps" agrees correctly with "she," but the intransitive verb has been given an object it does not subcategorize for).

Compositionality and generality. Feature-structure unification is a general-purpose mechanism: because it operates over arbitrary attribute-value pairs, it can express not just agreement but also, with enough features, elements of subcategorization itself (e.g., an attribute like COMPS listing the required complement categories, as HPSG does). Subcategorization frames, in their classical form, are more

narrowly purpose-built for describing valency and complement selection and do not, by themselves, provide a mechanism for propagating and checking shared features like number and person across a tree — that still requires a feature-passing/unification-like apparatus layered on top.

Where the constraint is checked. Subcategorization constraints are checked essentially locally, at the point where a head combines with its complements (a lexical, valency-driven check). Agreement constraints checked via feature structures can be local (subject NP and VP directly combining under S) but can also span longer distances in more complex constructions (e.g., agreement between a relative pronoun and its antecedent, or long-distance agreement in coordination structures), and unification-based feature propagation handles these naturally because features can be threaded up and down the derivation tree through the same unification mechanism used for local agreement.

Coverage of grammatical correctness overall. Because "grammatical correctness" in the scenario explicitly includes both subject–verb agreement and verb argument structures, neither mechanism alone is sufficient: subcategorization frames do not check agreement, and bare subcategorization frames (without an accompanying feature system) do not, by themselves, provide the fine-grained agreement-checking that unification-based feature structures provide. In practice, modern unification-based grammar formalisms (HPSG in particular) actually represent subcategorization requirements as a feature (a COMPS or SUBCAT list) inside a feature structure, so that a single unification-based apparatus enforces both agreement and correct argument structure. This is telling: it shows that feature structures are the more general, more expressive representational device, capable of subsuming subcategorization information as one more feature to be unified, while a bare subcategorization-frame approach cannot, in the reverse direction, subsume general agreement checking.

Example Illustration

Consider the developer's two target error types directly. For subject–verb agreement, take "The children plays quietly." A subcategorization-frame check on "plays" would only verify that "plays" has a subject and no illicit extra complement (which is satisfied here — "plays" intransitively-used is fine); it would not catch the fact that "children" (plural) does not agree with "plays" (singular), because the subcategorization frame does not encode or check the NUMBER feature. A feature-structure-based check, requiring NP.NUMBER = VP.NUMBER under unification, catches the error immediately: [NUMBER = plural] cannot unify with [NUMBER = singular].

For argument structure, take "She put the book." A feature-structure system focused purely on agreement features (person, number) would find nothing wrong — "she" and "put" agree correctly in person and number. A subcategorization-frame check on "put," which lexically requires an NP object and a locative PP ([_ NP PP]), correctly flags the sentence as incomplete because the obligatory locative complement ("on the table," "in the drawer," etc.) is missing. Neither error type is caught by the mechanism designed for the other, which is precisely the point.

Justification and Conclusion

Subcategorization frames and feature structures address two complementary and equally necessary aspects of grammatical correctness, and a system that needs to enforce both subject–verb agreement and correct verb argument structure genuinely needs both kinds of information somewhere in its grammar. That said, when asked which single representational device provides better overall support, feature structures are the stronger foundation, for two reasons. First, feature-structure unification is the more general mechanism: it can express agreement constraints natively, and — as demonstrated by formalisms like HPSG, where subcategorization is itself represented as a COMPS/SUBCAT feature inside the same feature structure —

it can also absorb argument-structure information as just another feature to unify, giving the grammar designer a single, uniform, compositional apparatus for checking both kinds of correctness rather than two separate, ad hoc mechanisms. Second, feature-structure unification propagates naturally through the derivation tree, which lets it enforce not only local agreement but also longer-distance and more intricate grammatical dependencies that a purely lexical, per-predicate subcategorization list is not designed to handle. For a grammar system whose goal is comprehensive enforcement of grammatical correctness — agreement and argument structure together — a unification-based feature-structure architecture that incorporates subcategorization information as one of its features is the more complete and better-supported design, with plain subcategorization frames serving as an important but narrower, complementary lexical component within that larger feature-based system.

5. A large-scale NLP system must perform fast and accurate dependency parsing on big datasets. The choice is between transition-based parsing and graph-based parsing. Analyze the differences between these methods in terms of decision-making and performance, and justify which is more suitable for large-scale applications.

Introduction

Large-scale NLP systems — web-scale information-retrieval pipelines, industrial search engines, and production systems that must syntactically analyze millions or billions of sentences — need dependency parsers that are not only accurate but also fast and predictable in their resource usage. Two dominant paradigms have emerged for data-driven dependency parsing: transition-based parsing, which builds a parse through a sequence of local, greedy (or beam-searched) actions, and graph-based parsing, which scores complete candidate dependency trees globally and searches for the highest-scoring one. This answer analyses how the two approaches differ in their decision-making process and their performance characteristics, and justifies which is more suitable for large-scale applications such as fast, accurate parsing over massive datasets.

Approach 1: Transition-Based Parsing — Working Principle

Transition-based parsing casts dependency parsing as a sequence of actions applied by a state machine, most commonly the arc-standard or arc-eager shift-reduce system built around a stack, a buffer of remaining input words, and a set of arcs constructed so far. At each step, a classifier (traditionally a linear classifier over hand-engineered features, more recently a neural network over word/context embeddings) looks at the current configuration — the top of the stack, the front of the buffer, and their neighbouring context — and predicts the next action: SHIFT (move the next word from the buffer to the stack), LEFT-ARC (create a dependency arc from the buffer/stack front to the stack top, and pop the stack top), or RIGHT-ARC (the symmetric operation creating an arc in the other direction). Parsing proceeds by repeatedly applying the predicted action until the buffer is empty and the stack contains only the root, at which point the sequence of arcs created constitutes the parse.

Because each decision is made locally — the classifier only looks at the current configuration, not the entire sentence at once — and because exactly one action is taken at each of the $O(n)$ steps needed to consume an n -word sentence (in the simplest greedy variant), transition-based parsing runs in linear time, $O(n)$, per sentence. This is its defining computational advantage. The trade-off is that each decision is made without full knowledge of how it will affect arcs built later in the sentence; a greedy transition-based parser commits to each action immediately and cannot revisit an earlier decision once made (a limited form of mitigation is possible via beam search, which keeps several candidate action sequences alive in

parallel and prunes low-scoring ones, trading a constant-factor increase in computation for improved accuracy by delaying some commitment).

Approach 2: Graph-Based Parsing — Working Principle

Graph-based parsing treats the sentence as a fully connected directed graph in which every word is a potential head of every other word, assigns a score to each candidate arc (again, traditionally via a linear model over engineered features, or more recently via a neural network that scores a head-dependent pair using their contextual embeddings), and then searches, over the exponentially large space of possible spanning trees of this graph, for the single spanning tree with the highest total arc score. For projective trees this is often done efficiently with dynamic-programming algorithms (e.g., Eisner's algorithm, $O(n^3)$); for the more general case of non-projective trees (where dependency arcs are allowed to cross, which happens in languages with freer word order), the Chu-Liu/Edmonds maximum spanning tree algorithm finds the globally optimal tree efficiently, in roughly $O(n^2)$ time for dense graphs.

Because graph-based parsing scores the tree as a whole (or, in first-order models, as a sum of independently scored arcs, with the search then finding the globally best combination of arcs subject to the tree constraint), it makes its decisions with a more complete, global view of the competing structural possibilities than a transition-based parser's local, step-by-step commitments. Higher-order graph-based models extend this further by scoring not just individual arcs but small combinations of arcs (siblings, grandparent-child triples), capturing structural interactions at the cost of increased algorithmic complexity, since exact inference for higher-order models is more expensive than the first-order $O(n^3)$ case.

Comparative Analysis

Nature of decision-making. Transition-based parsing makes a sequence of local, greedy (or beam-limited) decisions, committing to structure incrementally as it consumes the sentence left to right; it never explicitly scores or compares complete trees. Graph-based parsing makes one global decision, choosing among entire candidate trees (or, in practice, efficiently searching the space of trees via dynamic programming) based on a score that considers the tree as a whole, giving it a more holistic view of structural competition, particularly for long-distance dependencies where a transition-based parser's local context window may not "see" the relevant conditioning information at the moment a critical decision is made.

Time complexity. Transition-based parsing runs in linear time, $O(n)$, in its greedy form (or $O(n \cdot b)$ with a beam of width b), because it takes a fixed number of actions proportional to sentence length. Graph-based parsing is more expensive: $O(n^3)$ for first-order projective parsing via Eisner's algorithm, and higher-order variants scale worse still. For very large datasets — where an application must parse millions of sentences — this asymptotic difference is operationally significant: transition-based parsing scales essentially linearly with corpus size and sentence length, while graph-based parsing's per-sentence cost grows cubically (or worse) with sentence length, which matters for the occasional very long sentence and, in aggregate, for total processing throughput across a massive dataset.

Accuracy characteristics. Because it optimizes globally over the whole tree rather than committing to a sequence of unrecoverable local decisions, graph-based parsing has historically tended to produce more accurate results on long-distance dependencies and on sentences where an early local decision, if wrong, would otherwise propagate errors through the rest of the greedy parse (the classic error-propagation problem of pure greedy transition-based parsing). Modern neural transition-based parsers substantially narrow this accuracy gap by using rich contextual embeddings (so that even a "local" decision benefits from information about the whole sentence via the encoder) and by using beam search or dynamic-oracle

training to reduce error propagation, so the accuracy gap between the two approaches has shrunk considerably in recent neural systems, though a residual advantage for global, higher-order graph-based scoring on certain long-range phenomena is still commonly reported.

Handling of non-projectivity. Transition-based systems require special extensions (e.g., swap operations in the arc-eager system, or a pseudo-projective transformation of the training data) to correctly produce non-projective trees, since the basic stack/buffer mechanism naturally builds projective structures. Graph-based parsing with the Chu-Liu/Edmonds algorithm handles non-projective trees directly and efficiently as part of its core search, without additional machinery, which is an advantage for large multilingual systems that must correctly parse free-word-order languages at scale.

Engineering/production trade-offs. For an application whose primary requirement is throughput at massive scale (the stated scenario — "large-scale NLP system," "big datasets," and a need to justify "efficient approaches for processing massive linguistic datasets"), the linear-time behaviour of transition-based parsing translates directly into lower total compute cost and more predictable, easily parallelizable per-sentence latency, both of which are operationally critical when the system must process web-scale volumes of text within a fixed compute or time budget. Graph-based parsing's higher per-sentence complexity, while giving somewhat stronger structural guarantees and potentially higher raw accuracy in some cases, imposes a real and compounding cost when multiplied across millions or billions of sentences.

Example Illustration

Consider a search-engine pipeline that must dependency-parse the text of billions of indexed web pages to power syntax-aware search and question answering. If average sentence length is, say, 20 tokens, a transition-based parser performing roughly $2n = 40$ actions per sentence scales its total workload linearly with the number of sentences processed. A graph-based parser using Eisner's $O(n^3)$ algorithm performs work proportional to $20^3 = 8,000$ "units" per sentence for the same input — and because the exponent is cubic, sentences on the long tail of the length distribution (say, 60-token sentences, which are not unusual in web text) cost $60^3 = 216,000$ units under the graph-based approach versus a much more modest, linearly-scaled cost under the transition-based approach. At the scale of billions of documents, this difference compounds into a substantial gap in total compute time and infrastructure cost, directly affecting whether the pipeline can keep up with the ingestion rate of new content.

Conversely, consider a scenario prioritizing maximum syntactic accuracy on a comparatively small, high-value dataset — for example, parsing legal contracts or medical records where an error in a long-distance dependency (such as correctly attaching a modifier several clauses away from its head) could materially change downstream interpretation. Here, the smaller data volume makes the $O(n^3)$ cost of graph-based parsing entirely tractable, and its more global view of tree structure is worth the extra computation because accuracy, not throughput, is the binding constraint.

Justification and Conclusion

Transition-based and graph-based parsing represent a genuine speed-versus-global-optimality trade-off: transition-based parsing makes fast, local, incremental decisions in linear time but risks compounding local errors across the sentence, while graph-based parsing makes slower, globally optimized decisions over the whole candidate tree, at polynomial (cubic or worse) cost, generally producing marginally stronger structural accuracy, especially on long-distance dependencies and non-projective constructions. For the specific scenario described — a large-scale NLP system that must perform fast and accurate parsing on big datasets — transition-based parsing is the more suitable approach. Its linear time complexity, straightforward parallelizability across sentences and documents, and (in modern neural

implementations, using rich contextual encoders and beam search to mitigate error propagation) competitively high accuracy together make it the better fit for the throughput and scalability demands of large-scale processing. Graph-based parsing remains the preferable choice when the application can afford its higher per-sentence computational cost in exchange for the strongest possible global accuracy on a comparatively bounded volume of text, but as dataset size grows toward the "massive" scale described in the scenario, the linear scalability of transition-based parsing becomes the decisive, and ultimately more justifiable, factor.

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria Level	Scope of Items	Comparison Depth	Use of Evidence	Critical Thinking	Clarity of Presentation	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____